

Using torch.gather() for value-based RL in BBRL + filtering

Olivier Sigaud

Sorbonne Université

<http://www.isir.upmc.fr/personnel/sigaud>



Performing Bellman backup

- To update the critic network, we want to perform

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_{a \in A} Q(s_{t+1}, a) - Q(s_t, a_t)]$$

- The `compute_critic_loss()` function receives a tensor of Q-values of all actions through time and the actions of the agent at the same time
- How to get $Q(s, a)$ through time from these tensors?

time / Q-values	a_0	a_1	a_2	a_3	t	action
t_0	0.66	0.88	0.81	0.73	t_0	a_1
t_1	0.73	0.63	0.9	0.43	t_1	a_3
t_2	0.73	0.9	0.95	0.73	t_2	a_0
t_3	0.81	0.9	1.0	0.81	t_3	a_2
t_4	0.81	1.0	0.81	0.9	t_4	a_3
t_5	0.9	1.0	0.0	0.9	t_5	a_1

Using torch.gather(): the idea

- ▶ The answer is `torch.gather()`
- ▶ If life was simple, we would use `torch.gather(q-values, actions)...`

time / Q-values	a_0	a_1	a_2	a_3	t	action	$Q(s, a)$
t_0	0.66	0.88	0.81	0.73	t_0	a_1	0.88
t_1	0.73	0.63	0.9	0.43	t_1	a_3	0.43
t_2	0.73	0.9	0.95	0.73	t_2	a_0	0.73
t_3	0.81	0.9	1.0	0.81	t_3	a_2	1.0
t_4	0.81	1.0	0.81	0.9	t_4	a_3	0.9
t_5	0.9	1.0	0.0	0.9	t_5	a_1	1.0

- ▶ But life is not simple!

Using torch.gather(): the issue

- ▶ In practice, the actions Tensor is organized this way: $[a_1 \ a_3 \ a_0 \ a_2 \ a_3 \ a_1]$
- ▶ But we need it to be organized this way: $[[a_1][a_3][a_0][a_2][a_3][a_1]]$
- ▶ So, to get the adequate indexes, we need to use `actions.unsqueeze(-1)`
- ▶ Besides, `torch.gather()` produces a Tensor like this: $[[q_1][q_3][q_0][q_2][q_3][q_1]]$
- ▶ But we need a Tensor like this: $[q_1 \ q_3 \ q_0 \ q_2 \ q_3 \ q_1]$
- ▶ So we use `qvals = qvalues.gather(some params...).squeeze(-1)`

Using `torch.gather()`: `autoreset=False` or `True`

- ▶ The `gather` function takes two params: the dimension along which we want to gather data, and the tensor of indexes
- ▶ The dimension along which we want to gather data is actions.
- ▶ When `autoreset=False`, the Q-values tensor is a $(T \times E \times A)$ tensor: T for time, E for the number of envs, and A for actions
- ▶ Thus actions are at dimension 2
- ▶ So we use `qvals = qvalues.gather(dim=2, index=actions.unsqueeze(-1)).squeeze(-1)`
- ▶ When `autoreset=True`, the Q-values tensor is a $(2 \times (E \times A))$ tensor
- ▶ We consider the `qvalues[0]` tensor (i.e. time t) and actions are at dimension 1
- ▶ This time we use `qvals = torch.gather(qvalues[0], dim=1, index=actions.unsqueeze(-1)).squeeze(-1)`
- ▶ Note the use of `torch.gather()` instead of `qvalues.gather()`, both are equivalent

Reminder: get_transitions(): more details

- ▶ If $n_env > 1$, before `get_transitions()`: $[step_0^1 \ step_0^2 \dots \ step_0^{n_env}]$
- ▶ After `get_transitions()`, the vector is broken into pieces:
- ▶ Each key of the returned workspace has dimensions $[2, \ n_transitions, \ key_dim]$
- ▶ $key[0][0], key[1][0] = (step_1, step_2) \# \text{ for env 1}$
 $key[0][1], key[1][1] = (step_1, step_2) \# \text{ for env 2}$
 $key[0][2], key[1][2] = (step_2, step_3) \# \text{ for env 1}$
 $key[0][3], key[1][3] = (step_2, step_3) \# \text{ for env 2}$
 ...

Filtering terminated steps in autoreset=False

<i>Env₁</i>	done:	False	False	...	False	True	True	...	True
	cumulated reward:	0.8	1.2	2.4	3.8	3.8	3.8	...	3.8
<i>Env₂</i>	done :	False	False	...	True	True	True	...	True
	cumulated reward :	1.2	3.8	...	5.1	5.1	5.1	...	5.1
<i>Env₃</i>	done :	False	False	...	False	False	...	False	True
	cumulated reward :	1.7	4.0	...	5.1	6.3	...	9.2	9.2

- ▶ In autoreset=False, the tensors repeat the last info when done=True until all environments are done
- ▶ When processing tensors to compute gradient, we should not process the repeated info

<i>Env₁</i>	done:	False	False	...	False	True	True	...	True
	cumulated reward :	0.8	1.2	2.4	3.8	3.8	3.8	...	3.8
<i>Env₂</i>	done :	False	False	...	True	True	True	...	True
	cumulated reward :	1.2	3.8	...	5.1	5.1	5.1	...	5.1
<i>Env₃</i>	done :	False	False	...	False	False	...	False	True
	cumulated reward :	1.7	4.0	...	5.1	6.3	...	9.2	9.2

- ▶ So, in the code:
- ▶ `not_done = (~done[: -1]).int()`
- ▶ `td = my_calculation * not_done`

Any question?



Send mail to: Olivier.Sigaud@upmc.fr